

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Corso di Laurea Triennale in Informatica

Sviluppo di strategie RAG per la generazione di codice sicuro

Relatore:

Fabio Palomba

Candidato:

Alessandro Bonomo

Mat. 0512123590

Correlatore:

Matteo Cicalese

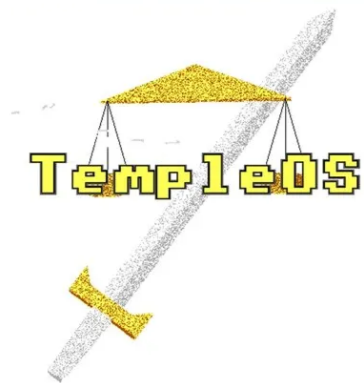
ANNO ACCADEMICO 2025/2026

Questa tesi è stata realizzata nel



"An idiot admires complexity, a genius admires simplicity"

Terry A. Davis



Contents

1	Introduzione	6
2	Stato dell'arte	7
2.1	Large Language Models per la Generazione di Codice	7
2.2	Vulnerabilità del Codice Generato da LLM	7
2.3	Tecniche di Mitigazione Esistenti	8
2.3.1	Prompt Engineering / Chain of thought	8
2.3.2	Retrieval-Augmented Generation (RAG)	8
2.3.3	Architettura RAG	8
2.3.4	Vantaggi della RAG	8
2.3.5	Tecniche di Chunking	9
2.4	Vector Databases	9
2.5	RAG per Code Security	9
2.6	Sistemi Esistenti	9
2.6.1	Elysia	9
2.6.2	TRON vs Elysia	10
3	TRON: un agente RAG per la generazione di codice sicuro	11
3.1	Domande di Ricerca e Strategia di Validazione	11
3.2	Definizione delle Metriche di Valutazione	12
3.2.1	Fix Rate (Tasso di Correzione)	12
3.2.2	Modello Generativo: Gemini 2.5 Pro	13
3.3	Approccio Metodologico	13
3.4	Architettura di TRON	14
3.4.1	Il core agentic TRON	14
3.4.2	Le altre componenti connessi al core	17
3.5	Flusso di lavoro in TRON	20
3.5.1	Sperimentazione del flusso di lavoro	20
4	Analisi dei risultati	23
4.1	Caso Studio 1: Stack-based Buffer Overflow (CWE-787)	23
4.2	Caso Studio 2: Integer Overflow (CWE-190)	25
4.3	Caso Studio 3: Double Free (CWE-416) - Estensione	26
5	Sintesi Quantitativa dei Risultati	26

CONTENTS

5.1	Contributi rispetto allo stato dell'arte	27
5.2	Limitazioni identificate	28
6	Conclusioni	28
6.1	Impatto e sviluppi futuri	28
7	Ringraziamenti	30
	References	31

Abstract

L'adozione crescente dei Large Language Models (LLM) ha rivoluzionato il modo in cui scriviamo il codice, è sempre più raro che ci sia un programmatore che scrive codice senza il boost di produttività che l'intelligenza artificiale fornisce. Questo boost in produttività premia nel breve termine dando soluzioni all'apparenza funzionanti e robuste ma, nel lungo termine, si ha un peggioramento della qualità e della sicurezza del codice. L'AI non ha una visione progettuale a lungo termine del progetto ed è già capitato che applicazioni scritte dai cosiddetti vibecoders (termine abbastanza diffuso per indicare coloro che delegano l'intero processo di scrittura, analisi e sviluppo del codice all'IA) avessero gravi falle informatiche che hanno portato a gravi conseguenze per gli utenti e per gli sviluppatori.

Le attuali strategie di mitigazione presentano criticità sostanziali. Il prompt engineering soffre di inconsistenza e richiede un'expertise specifica da parte dell'operatore, mentre il fine-tuning dei modelli risulta computazionalmente oneroso e difficile da mantenere aggiornato rispetto all'evoluzione quotidiana delle minacce (CVE). Parallelamente, gli strumenti di analisi statica (SAST) come CodeQL eccellono nell'identificazione delle vulnerabilità ma mancano della capacità generativa necessaria per guidare lo sviluppatore verso una risoluzione contestualizzata e sicura

Questo lavoro di tesi presenta TRON (Trace-ON), un sistema basato su architettura Retrieval-Augmented Generation (RAG) progettato per tracciare errori e migliorare la sicurezza del codice C generato da LLM. TRON integra un database vettoriale Weaviate per l'indicizzazione e il recupero di documentazione sulla sicurezza nel linguaggio di programmazione C e altre best practice per la programmazione sicura. TRON si collega direttamente al repository github ed è in grado di leggere gli alert generati da CodeQL ¹, in questo modo ha la visione completa delle vulnerabilità di cui soffre la code base. Inoltre, è sempre possibile fornire snippet di codice indipendenti dalla codebase per farne l'analisi.

¹<https://codeql.github.com/>

1 Introduzione

L'utilizzo di LLM per la generazione di codice offre vantaggi innegabili in termini di produttività e accessibilità alla programmazione, permettendo anche a sviluppatori meno esperti di produrre codice funzionante rapidamente. Tuttavia, questi benefici si accompagnano a rischi significativi: gli LLM apprendono da repository pubblici che contengono inevitabilmente codice vulnerabile, e la loro natura statistica li porta a replicare pattern comuni senza comprensione semantica dei rischi di sicurezza. Inoltre, la conoscenza incorporata nei modelli diventa rapidamente obsoleta rispetto all'evoluzione delle minacce e delle best practices di sicurezza. Gli approcci esistenti per mitigare questi rischi presentano limiti sostanziali. Il prompt engineering richiede expertise specifica e non garantisce risultati consistenti. Il fine-tuning dei modelli è costoso computazionalmente e richiede aggiornamenti frequenti per mantenere la rilevanza. Gli strumenti di analisi statica come CodeQL sono eccellenti nell'identificazione di vulnerabilità ma offrono supporto limitato nella loro risoluzione: segnalano il problema senza fornire guidance contestuale su come correggerlo mantenendo la funzionalità desiderata. TRON mira a colmare questo gap combinando i punti di forza di diversi approcci: sfrutta la capacità generativa degli LLM mantenendoli ancorati a documentazione di sicurezza verificata attraverso RAG, integra l'analisi statica di CodeQL per una visione completa delle vulnerabilità reali del progetto, e fornisce non solo identificazione ma anche guidance prescrittiva per la correzione. A differenza di sistemi generici come Elysia, TRON è specializzato sulla sicurezza del codice C e opera con database locale, permettendo un controllo completo sui documenti di riferimento e garantendo privacy per progetti sensibili.

2 Stato dell'arte

2.1 Large Language Models per la Generazione di Codice

I Large Language Models hanno rivoluzionato il campo dell'elaborazione del linguaggio naturale e, più recentemente, della generazione automatica di codice. Modelli come Grok, GPT-4, Claude, Copilot e Code Llama hanno dimostrato capacità impressionanti nel comprendere richieste in linguaggio naturale e tradurle in codice funzionante in molteplici linguaggi di programmazione. Questi modelli sono tipicamente addestrati su enormi corpus di testo che includono repository di codice pubblici (come GitHub), documentazione tecnica, forum di discussione (Stack Overflow) e libri di testo. L'architettura Transformer alla base di questi modelli permette di catturare dipendenze a lungo raggio nel codice e di generare soluzioni contestualmente appropriate. Studi recenti hanno dimostrato che questi modelli possono raggiungere tassi di successo superiori al 70% su benchmark [1] di programmazione competitiva come HumanEval e MBPP, evidenziando il loro potenziale come assistenti alla programmazione. Tuttavia, la natura stessa del training introduce rischi intrinseci. I repository pubblici contengono inevitabilmente sia codice eccellente che codice vulnerabile, obsoleto o semplicemente errato. Gli LLM, apprendendo pattern statistici da questi dati, possono replicare e propagare vulnerabilità di sicurezza senza alcuna comprensione semantica del rischio che stanno introducendo.

2.2 Vulnerabilità del Codice Generato da LLM

Un esempio recente è stato l'incidente con l'app TEA [2] in cui, nel luglio 2025, oltre 200.000 utenti hanno subito una violazione dei dati personali. L'applicazione, sviluppata rapidamente utilizzando strumenti basati sull'intelligenza artificiale, presentava vulnerabilità critiche: una sezione riservata agli sviluppatori era accessibile senza autenticazione, le chiavi segrete per i servizi cloud erano esposte pubblicamente, e una semplice modifica numerica nell'URL permetteva di accedere ai dati di altri utenti (vulnerabilità IDOR). Nomi, email, password, foto e persino le chiavi interne del sistema sono finiti nelle mani di malintenzionati, dimostrando che l'intelligenza artificiale deve essere uno strumento di supporto per la scrittura del codice e non un sostituto. La letteratura scientifica ha documentato sistematicamente questi problemi. Pearce et al. [3] hanno condotto uno studio su GitHub Copilot dimostrando che circa il 40% del codice generato per scenari di sicurezza conteneva vulnerabilità CWE (Common Weakness Enumeration). Allo stesso modo, analisi su modelli open-source come Code Llama hanno rilevato che, sebbene questi sistemi eccellano in task di sintassi e logica base, mostrano lacune significative nella generazione di codice sicuro per linguaggi low-level come C e C++,

[4] dove la gestione manuale della memoria e i comportamenti undefined possono portare a vulnerabilità critiche.

2.3 Tecniche di Mitigazione Esistenti

Diverse strategie sono state proposte per migliorare la sicurezza del codice generato da LLM, ciascuna con vantaggi e limitazioni specifiche.

2.3.1 Prompt Engineering / Chain of thought

Il prompt engineering consiste nel raffinare le istruzioni fornite all'LLM per guidarlo verso output più sicuri. La ricerca ha esplorato diverse tecniche: la specificazione esplicita di requisiti di sicurezza ("genera codice sicuro contro buffer overflow"), il role prompting che assegna al modello il ruolo di esperto di sicurezza, il few-shot learning con esempi di codice sicuro e insicuro, e il chain-of-thought prompting [5] che incoraggia il modello a ragionare passo-passo sui rischi di sicurezza. Tuttavia, l'efficacia del prompt engineering dipende fortemente dall'esperienza dell'utente e dalla formulazione specifica delle richieste.

2.3.2 Retrieval-Augmented Generation (RAG)

La Retrieval-Augmented Generation è un paradigma architetturale per migliorare le capacità dei Large Language Models in task knowledge-intensive. Introdotto da Lewis et al. nel 2020,[6] RAG combina un sistema di information retrieval con un modello generativo, permettendo di arricchire dinamicamente il contesto di generazione con informazioni recuperate da knowledge base esterne.

2.3.3 Architettura RAG

Un sistema RAG tipico è composto da tre componenti principali: il Document Processing Pipeline, responsabile dell'ingestion, chunking ed embedding dei documenti; il Vector Database, che memorizza gli embeddings e permette ricerche per similarità semantica; e la Generation Pipeline, che recupera documenti rilevanti e li fornisce come contesto aggiuntivo all'LLM durante la generazione.

2.3.4 Vantaggi della RAG

I principali vantaggi dell'approccio RAG rispetto a fine-tuning o prompt engineering sono documentati estensivamente in letteratura [6]. La conoscenza rimane aggiornabile poiché i documenti possono essere aggiunti o modificati senza riaddestramento del modello. La tracciabilità è garantita attraverso citazioni alle fonti utilizzate, aumentando la fiducia nelle risposte generate. Le allucinazioni vengono ridotte significativamente attraverso l'ancoraggio a documenti reali. RAG permette inoltre specializzazione domain-specifica di LLM

general-purpose senza training, ed evita i costi computazionali del fine-tuning mantenendo il modello base.

2.3.5 Tecniche di Chunking

La suddivisione dei documenti in chunks è un passo critico nelle pipeline RAG [7]. La ricerca ha esplorato diverse strategie: chunking character-based con divisione a intervalli fissi, sentence-based che rispetta i confini naturali delle frasi, token chunking basato sulla lunghezza dei token dell'LLM, e overlapping chunks con sovrapposizione per preservare il contesto tra segmenti adiacenti. La scelta della strategia e della dimensione ottimale dei chunks impatta significativamente sulla qualità del retrieval e, conseguentemente, sulla generazione. Studi empirici suggeriscono che chunks di 512-1024 token con overlap offrono un ottimo bilanciamento tra contesto preservato e precisione del retrieval. TRON usa il token chunking by default, ottimizzato per documentazione tecnica di sicurezza [8].

2.4 Vector Databases

I database vettoriali sono specializzati nell'indicizzazione e ricerca efficiente di embeddings ad alta dimensionalità. Weaviate, database vettoriale open-source, offre supporto per multiple modalità e permette query complesse con filtri metadata. Pinecone fornisce un servizio gestito per vector search scalabile. Questi sistemi utilizzano algoritmi come HNSW (Hierarchical Navigable Small World) [9] o IVF (Inverted File) per accelerare la ricerca nearest-neighbor in spazi ad alta dimensionalità.

2.5 RAG per Code Security

L'applicazione di RAG alla generazione di codice sicuro è un'area emergente di ricerca. L'idea fondamentale è quella di arricchire il contesto di generazione con best practices di sicurezza, pattern di codice sicuro e documentazione rilevante recuperata dinamicamente. Lavori recenti hanno esplorato l'utilizzo di RAG per recuperare esempi di patch di sicurezza da database e ne confermano l'efficacia [10].

2.6 Sistemi Esistenti

2.6.1 Elysia

Elysia è un sistema RAG con Chain of thought e feedback loop sviluppato da Weaviate ² che fornisce un'architettura completa per applicazioni question-answering. Include un frontend React e backend FastAPI, con supporto nativo per Weaviate come vector database.

Tuttavia, Elysia presenta limitazioni:

²<https://github.com/weaviate/elysia>

- Database in cloud
- Mancanza di processing per pdf strutturati e pdf con testo in immagini/scansioni

2.6.2 TRON vs Elysia

TRON è un fork di Elysia ³ quindi conserva la strategia RAG di chain of thought, feedback loop e aggiunge la possibilità di leggere vulnerabilità su repository remoti scansionati da CodeQL. Inoltre, è dotato di Database vettoriale locale (invece che in cloud) e di un nuovo componente chiamato "DocProcessor" che effettua il chunking, embedding e caricamento di PDF nel database vettoriale.

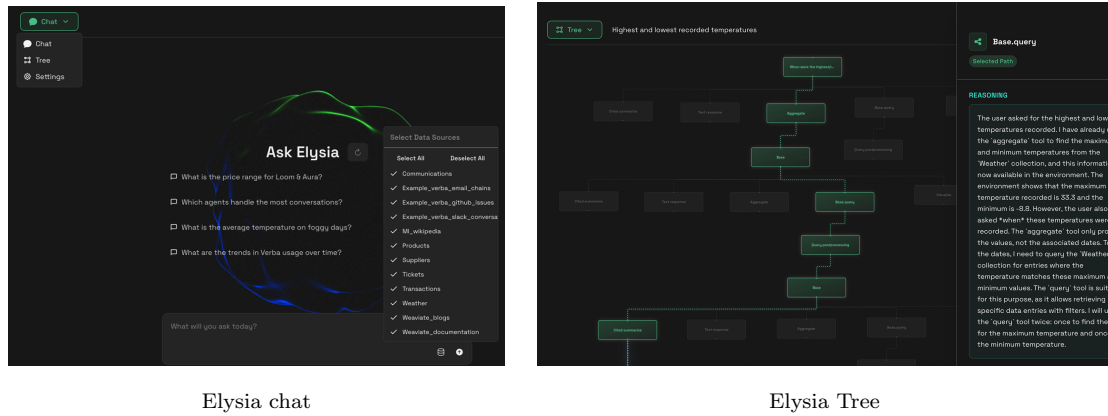


Figure 1: Elysia comparison

³<https://github.com/AlessandroBonomo28/TRON>

3 TRON: un agente RAG per la generazione di codice sicuro

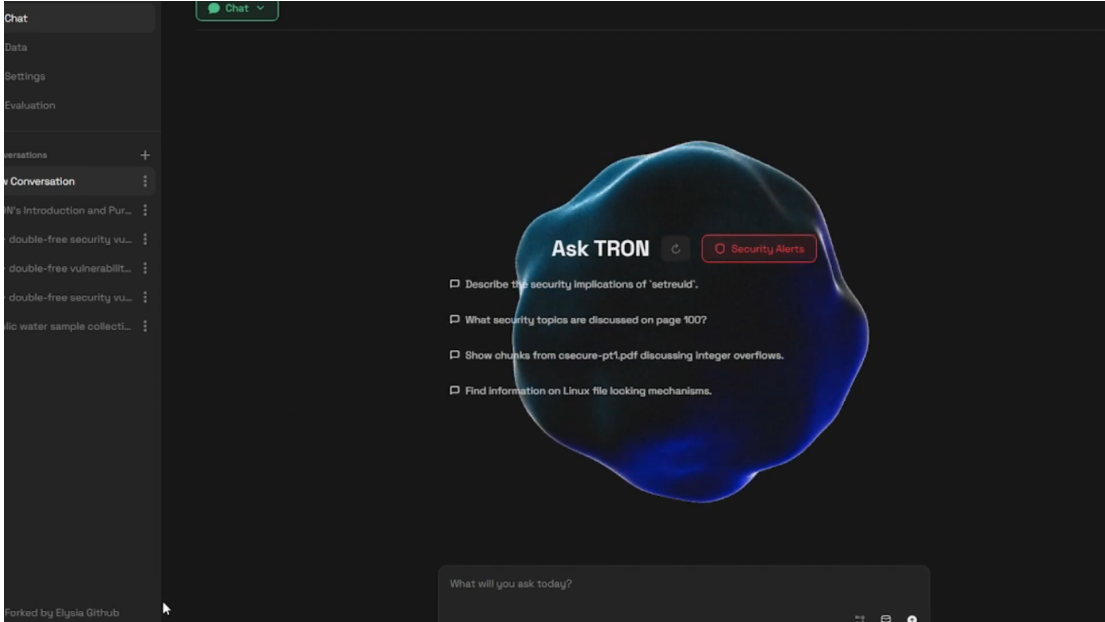


Figure 2: TRON chat con opzione "Security Alerts"

3.1 Domande di Ricerca e Strategia di Validazione

Per valutare rigorosamente l'efficacia dell'architettura TRON, la ricerca è guidata da due Research Questions (RQ) che mirano a quantificare il valore aggiunto dell'approccio RAG rispetto all'uso standard degli LLM.

- **RQ₁: Efficacia Comparativa (Baseline vs. TRON)**

In che misura l'integrazione di una pipeline RAG agentica, supportata da documentazione di sicurezza specifica, migliora la capacità del sistema di correggere le vulnerabilità rispetto a un'interazione standard con un LLM?

Per rispondere a questa domanda, è stato definito un protocollo sperimentale che confronta due modalità di interazione distinte, mantenendo costante il modello sottostante per isolare l'effetto della variabile "RAG":

1. **Baseline (Gruppo di Controllo):** Rappresenta l'approccio "naïve" o standard. Utilizza il modello Gemini 2.5 Pro in modalità Zero-Shot o Few-Shot generico. Al modello viene fornito il codice vulnerabile e una richiesta di correzione generica

(“Fix this code”), senza accesso alla Knowledge Base vettoriale e senza i dettagli strutturati dell’alert CodeQL. Questo simula l’esperienza tipica di uno sviluppatore che usa ChatGPT o Copilot senza un contesto di sicurezza specializzato.

2. **TRON (Gruppo Sperimentale):** Utilizza lo stesso modello Gemini 2.5 Pro, ma all’interno dell’architettura TRON. Il prompt viene costruito dinamicamente includendo:

- Il codice vulnerabile.
- I metadati precisi dell’alert CodeQL (estratti tramite API).
- I “chunk” di documentazione più rilevanti recuperati dal database vettoriale Weaviate.

La comparazione tra Baseline e TRON permetterà di determinare se il miglioramento nella qualità del codice è attribuibile alla conoscenza recuperata (RAG) piuttosto che alle capacità intrinseche del modello.

• **RQ₂: Tasso di Successo Operativo (Fix Rate)**

Qual è il tasso di successo (Fix Rate) del sistema TRON nel risolvere automaticamente le vulnerabilità identificate dagli alert di CodeQL senza introdurre regressioni?

Questa domanda si focalizza sull’affidabilità del sistema. Non è sufficiente che il codice sembri corretto; deve essere funzionalmente valido e sicuro secondo i criteri dell’analisi statica.

3.2 Definizione delle Metriche di Valutazione

Per trasformare l’analisi qualitativa in quantitativa, definiamo formalmente le metriche utilizzate nello studio.

3.2.1 Fix Rate (Tasso di Correzione)

Il Fix Rate è la metrica primaria per valutare la performance di TRON. Il Fix Rate è definito come la percentuale di vulnerabilità identificate che vengono transite dallo stato *Open* allo stato *Fixed* (o *Closed*) a seguito dell’intervento del sistema.

La formula utilizzata è:

$$\text{Fix Rate} = \frac{N_{\text{fixed}}}{N_{\text{total}}} \times 100 \quad (1)$$

Dove:

- N_{total} è il numero totale di vulnerabilità (alert CodeQL) sottoposte al sistema.
- N_{fixed} è il numero di vulnerabilità per le quali la patch generata soddisfa simultaneamente tre criteri:

1. **Validità Sintattica:** Il codice compilato non genera errori di compilazione.
2. **Validità di Sicurezza:** Una successiva scansione con CodeQL non rileva più l'alert originale.
3. **Integrità Funzionale:** Il codice mantiene la logica originale senza introdurre comportamenti indefiniti o regressioni (verificata tramite analisi manuale o unit test nel contesto sperimentale).

3.2.2 Modello Generativo: Gemini 2.5 Pro

Il core di TRON è basato sul modello **Gemini 2.5 Pro** (accessibile via Vertex AI). Questo modello è stato selezionato per la sua estesa finestra di contesto e le superiori capacità di *reasoning* rispetto alle iterazioni precedenti.

Per garantire la generazione di codice sicuro, minimizzando le allucinazioni sintattiche a favore del determinismo, sono stati configurati i seguenti iperparametri:

- **Temperature: 0.2.** Valore basso per massimizzare l'aderenza alle specifiche e la precisione sintattica, riducendo la "creatività" del modello.
- **Top-P (Nucleus Sampling): 0.95.** Filtra le probabilità meno verosimili mantenendo coerenza logica.
- **Top-K: 40.** Limita la selezione ai 40 token più probabili (default standard per task tecnici su modelli recenti).
- **Max Output Tokens: 8192.** Garantisce che le patch e le relative spiegazioni non vengano troncate anche in scenari complessi.

3.3 Approccio Metodologico

Il lavoro ha seguito un processo suddiviso in più fasi:

1. **Analisi del Problema:** ho identificato i limiti degli assistenti di codifica attuali e i potenziali rischi di sicurezza che comportano.
2. **Progettazione di TRON:** ho definito un'architettura che integra recupero di informazioni contestuali (RAG), logica agentica e analisi statica del codice. Per costruire il sistema ho modificato il framework Elysia, aggiungendo funzionalità per embedding in locale e l'elaborazione di documenti complessi.
3. **Sviluppo:** ho implementato i componenti principali del sistema: il DocProcessor per l'elaborazione dei documenti, un database vettoriale Weaviate in locale e un modulo per connettersi a repository remoti git e interpretare i report di vulnerabilità.

3.4 Architettura di TRON

TRON ha un'architettura modulare che si può schematizzare con questa immagine:

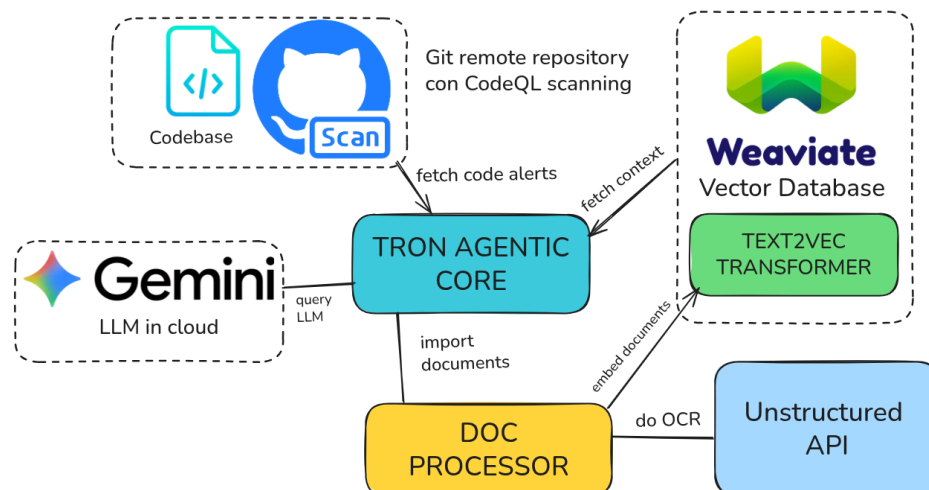


Figure 3: Architettura di TRON

3.4.1 Il core agentico TRON

Rappresenta il nucleo centrale dell'architettura ("TRON AGENTIC CORE"). Questo componente agisce da orchestratore principale collegando tutti i servizi esterni: interroga l'LLM, recupera il contesto dal database vettoriale, riceve gli alert di sicurezza dal repository e gestisce il flusso di importazione dei documenti attraverso il Doc Processor. Inoltre, basandosi su elysia eredita la sua architettura agentica

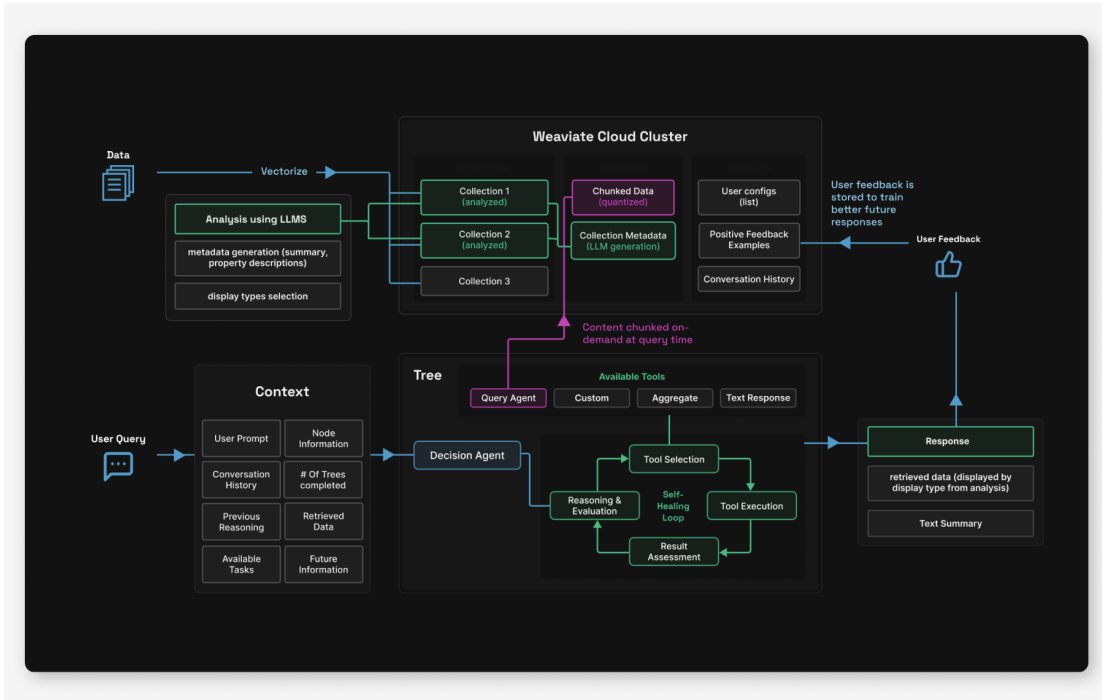


Figure 4: Architettura agent

Orchestrazione Intelligente

Ogni nodo dell'albero è orchestrato da un **Agente Decisionale** dotato di una consapevolezza globale del contesto (*Global Context Awareness*). L'agente valuta l'ambiente, le azioni disponibili, le azioni passate e quelle future per strategizzare lo strumento migliore da utilizzare .

Reasoning Handoff (Passaggio del Ragionamento)

L'agente non produce solo un output funzionale, ma esplicita il suo ragionamento. Questo "pacchetto cognitivo" viene tramandato agli agenti successivi, garantendo che ogni componente lavori verso lo stesso obiettivo con piena consapevolezza delle intenzioni precedenti .

Self-Healing e Gestione Errori

La struttura ad albero abilita meccanismi avanzati di auto-guarigione:

- **Impossible Flag:** Se l'agente determina che un compito non può essere completato con i dati disponibili (es. cercare prezzi di pantaloni in una collezione di gioielli), imposta un "flag di impossibilità" invece di allucinare una risposta .
- **Adattamento alla Ricerca:** Se una query produce risultati irrilevanti, il sistema non fallisce ma decide autonomamente di riprovare con termini di ricerca diversi o

filtri meno rigorosi .

- **User feedback:** una volta prodotta la risposta, l'utente può fornire feedback con un "like" o "dislike" per personalizzare e migliorare le risposte del modello.

Osservabilità (Glass Box)

Il sistema offre trasparenza totale visualizzando l'attraversamento dell'albero in tempo reale, permettendo agli utenti di osservare il ragionamento dell'LLM all'interno di ogni nodo.

Visualizzazione Dinamica delle Fonti Dati Mentre molti assistenti IA si limitano a risposte testuali, TRON eredita la capacità di scegliere dinamicamente come visualizzare i dati in base al contesto e al contenuto.

Formati Adattivi: Il sistema supporta nativamente molteplici formati di visualizzazione: display generici, tabelle, schede prodotto e-commerce, ticket (es. GitHub), conversazioni/messaggi, documenti e grafici .

Selezione Intelligente: Prima di utilizzare qualsiasi strumento, un LLM analizza la struttura delle collezioni dati (campionamento, controllo campi, generazione sommari). Basandosi su questa analisi, raccomanda il formato di visualizzazione più appropriato per quella specifica tipologia di dati. L'utente mantiene comunque la possibilità di mappare manualmente queste visualizzazioni.

TRON come Esperto Automatico dei Dati I sistemi RAG "ingenui" (*Naive RAG*) spesso falliscono con dati complessi o ambigui perché operano con una "ricerca cieca", senza comprendere l'ambiente in cui lavorano. TRON risolve questo problema diventando un esperto automatico del dominio dati .

Analisi Pre-Esecuzione: Al momento della connessione con il cluster vettoriale (Weaviate), un LLM analizza le collezioni per esaminare la struttura dei dati, creare riassunti e generare metadati ricchi. Questa conoscenza pregressa è essenziale per gestire query complesse che i sistemi tradizionali non saprebbero interpretare .

Metadati Semantici: La generazione di metadati permette al sistema di capire il *significato* e la *struttura* dei dati prima di eseguire query vettoriali, evitando ricerche "al buio" che sperano semplicemente nella rilevanza casuale.

Data Explorer e Dashboard: La piattaforma include un esploratore dati completo con ricerca BM25, ordinamento e filtri automatici (es. range min/max per dati numerici). La dashboard offre una visione d'insieme delle collezioni e permette agli utenti di modificare i metadati e i sommari generati dall'IA, riconoscendo che gli LLM, pur potenti, necessitano

talvolta di supervisione umana.

3.4.2 Le altre componenti connessi al core

Il Database vettoriale Weaviate e Embedder: Il sistema utilizza **Weaviate** come Vector Database. Al suo interno integra il modulo **TEXT2VEC TRANSFORMER**, che si occupa di ricevere i documenti processati per la vettorializzazione. Il database fornisce il contesto necessario al Core TRON per arricchire le risposte.

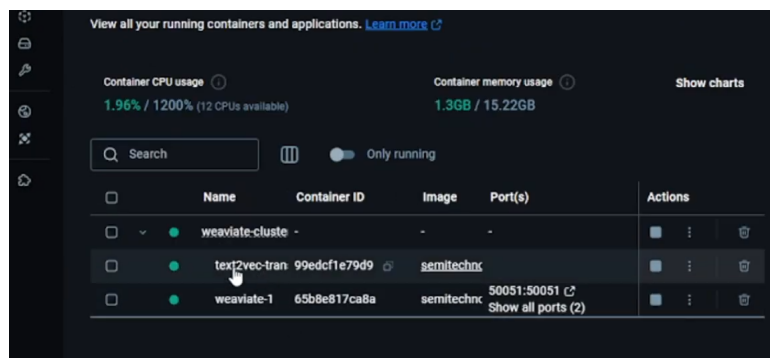


Figure 5: Weaviate DB e Text2vec transformer in esecuzione in docker container

Il modulo di Ingestion DocProcessor: Il **DOC PROCESSOR** è il modulo dedicato alla gestione dei documenti, che, ad oggi, non è presente in Elysia e rappresenta una innovazione. È collegato al Core TRON per le operazioni di importazione ("import documents") e si interfaccia direttamente con il database Weaviate per inviare i documenti da vettorializzare ("embed documents"). Gestisce inoltre il flusso verso i servizi di OCR.

```
(env) PS C:\Users\alex\Desktop\programming\tirocinio\elysia\docprocessor> docprocessor start
149962, 151013, 152947, 156318, 159696, 162724, 165567, 168284, 172283, 175172,
179020, 181363, 184425, 187818, 190362, 193960, 196389, 199873, 204036, 208578,
212752, 216773, 221166, 225303, 228217, 231835, 236817, 240533, 245435, 249551,
253719, 257467, 260846, 264690, 267225, 268697, 270775, 274315, 277430, 281772,
285225, 288521, 291371, 295261, 299757, 302122, 305756, 308529, 312868]
i Unstructured Pages []
Chunking documents: 100%|████████████████████████████████████████| 1/1 [00:00<00:00, 13.84it/s]
✓ Chunking completed
i (1/1) Importing document csecure-pt2.pdf with 36 batches
Importing batches: 100%|████████████████████████████████████████| 36/36 [01:24<00:00, 2.35s/it]
✓ Embedding successful
INFO: 127.0.0.1:13077 - "POST /api/load_data HTTP/1.1" 200 OK
```

Figure 6: Backend del docprocessor in azione

3 TRON: UN AGENTE RAG PER LA GENERAZIONE DI CODICE SICURO

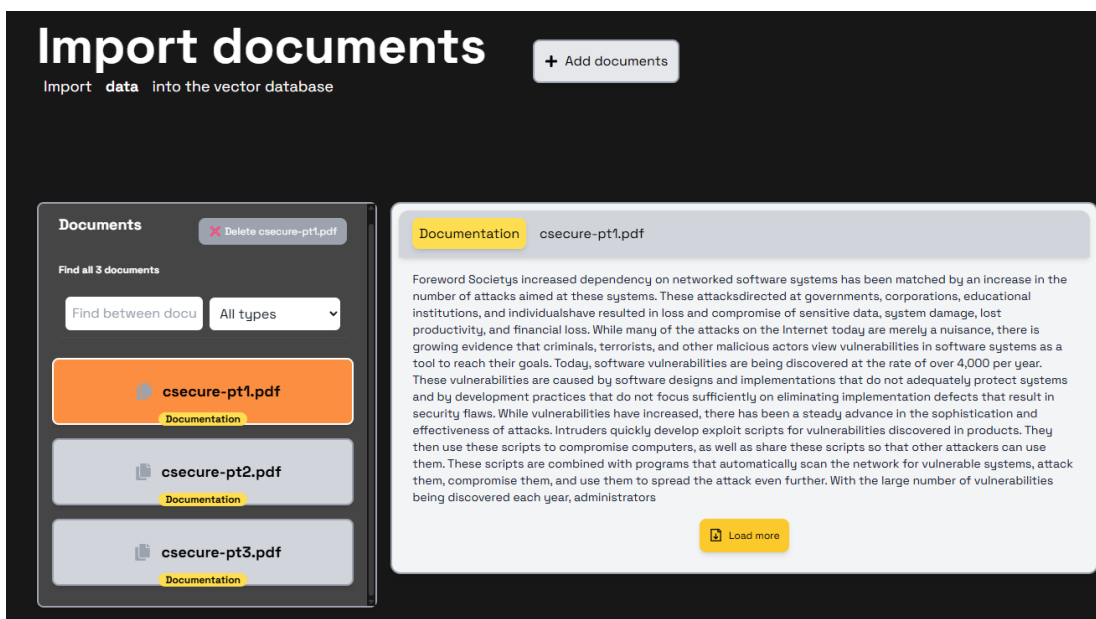
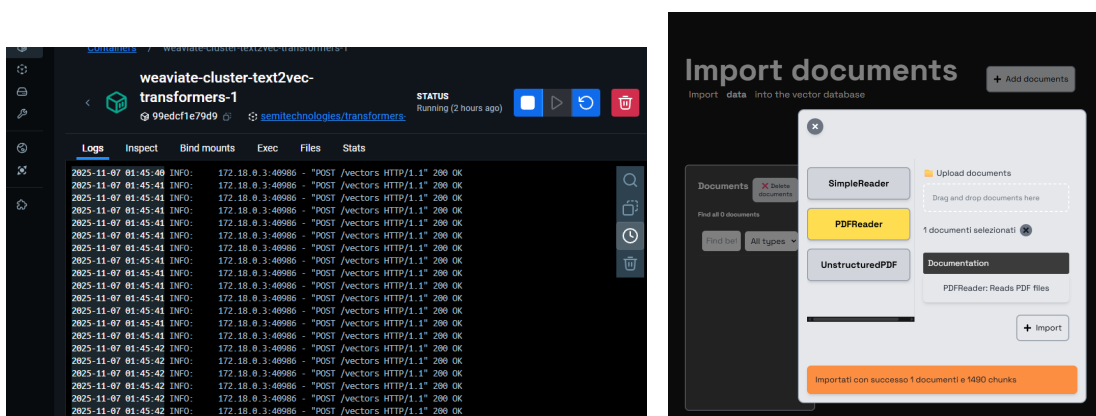


Figure 7: UI integrata in TRON per l'utilizzo del docprocessor

Il Modulo OCR per scansione di pdf con immagini: Modulo per i documenti non strutturati che richiedono riconoscimento ottico, il Doc Processor delega l'OCR alla **Unstructured API**, permettendo l'estrazione di testo da file complessi prima dell'embedding. L'Unstructured API può essere hostata in locale in un docker container oppure utilizzare l'API cloud.



Docker logs dell'embedder durante processo di importazione

Possibilità di selezionare PDF non strutturati

Figure 8: Importazione di documenti con docprocessor

3 TRON: UN AGENTE RAG PER LA GENERAZIONE DI CODICE SICURO

Il modulo di code scanning: Github CodeQL: Il sistema monitora un **Git remote repository** (Codebase) integrato con **CodeQL scanning**. In TRON è possibile collegare un repository remoto ed effettuare il fetch dei code alerts, permettendo all'agente di avere visibilità sulla qualità e sicurezza del codice.

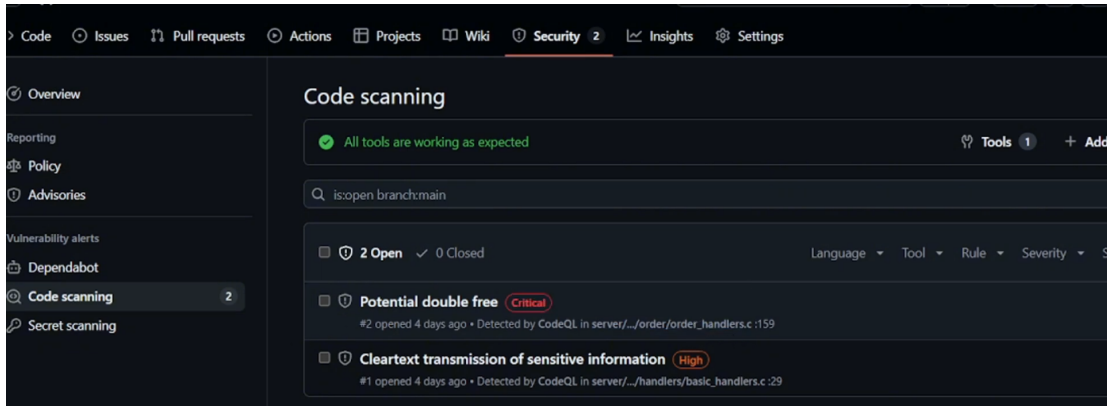


Figure 9: Esempio di alert generati da CodeQL in un mio repository remoto Github

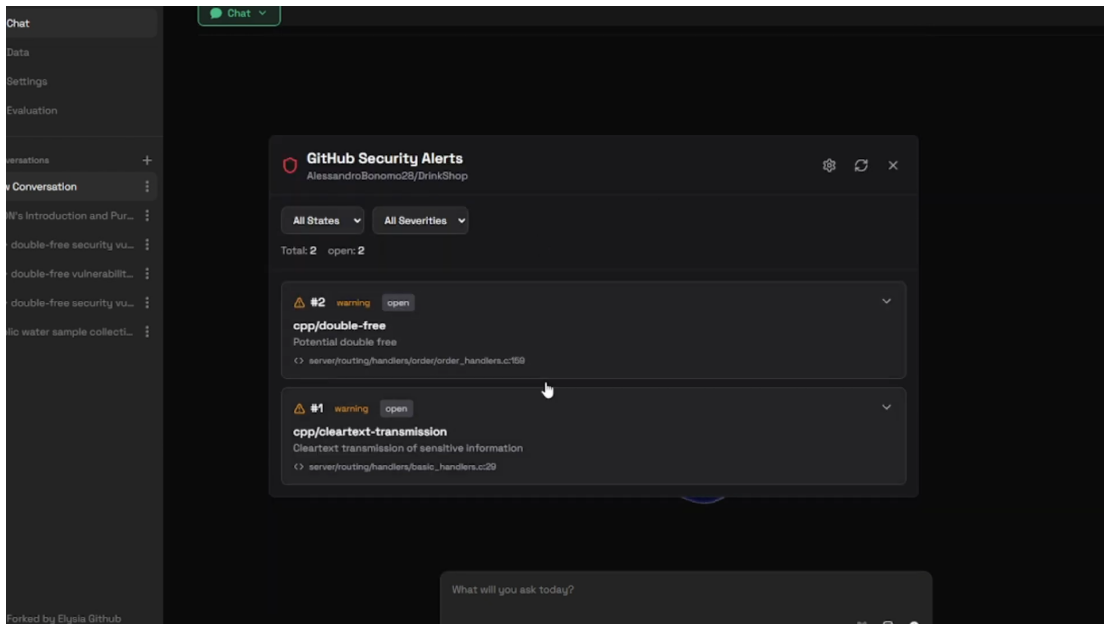


Figure 10: Gli stessi alert visualizzati all'interno di TRON

LLM in cloud Gemini: Il modello linguistico utilizzato è **Gemini** ("LLM in cloud"). Viene interrogato direttamente dal Core TRON ("query LLM") per le attività di ragionamento e generazione del linguaggio naturale.

3.5 Flusso di lavoro in TRON

TRON genera codice sorgente sicuro rispettando il seguente workflow:

1. L'utente collega il proprio repository github a TRON (deve farlo solo la prima volta).
2. L'utente collega carica documentazione relativa al linguaggio di programmazione di cui vuole rendere esperto TRON.
3. Ora occorre effettuare l'analisi della documentazione importata per ricavare i metadati
4. Ora che TRON ha il knowledge necessario per conoscere le vulnerabilità del linguaggio e può effettuare il fetch delle vulnerabilità rilevate dall'analisi statica di CodeQL, è possibile porre domande generiche (TRON risponderà utilizzando solo la Knowledge Base per il contesto teorico) oppure selezionare un particolare alert da analizzare (TRON risponderà usando sia la Knowledge Base, sia i dati specifici relativi all'alert per una risposta mirata).

3.5.1 Sperimentazione del flusso di lavoro

Ora riporto un esempio di utilizzo di TRON per correggere una vulnerabilità in un mio vecchio progetto C. Come prima cosa ho importato un libro di buone pratiche di sicurezza in C nella knowledge base di TRON. Successivamente, effettuo l'analisi in TRON per estrarre i metadati:

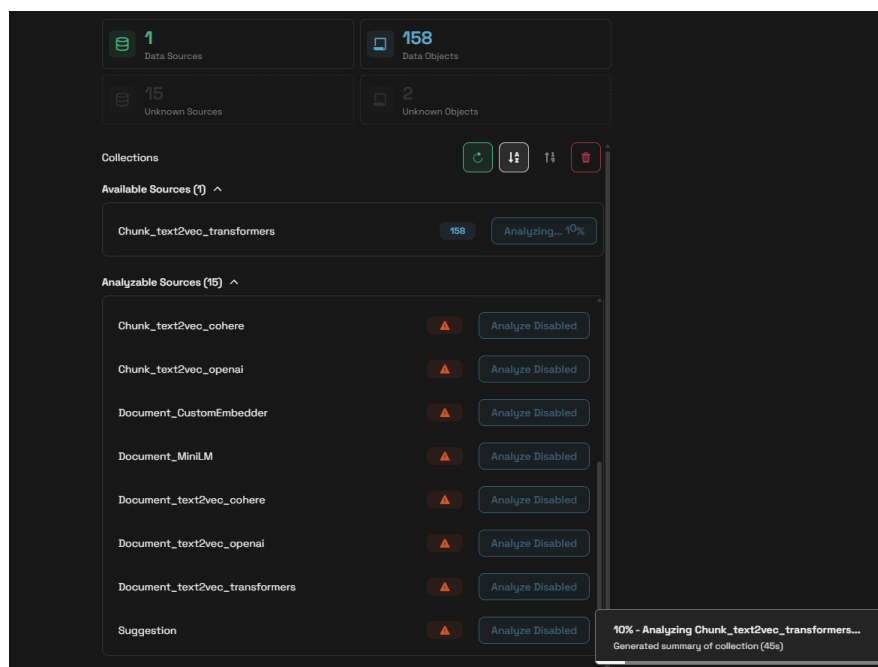


Figure 11: Analisi dei chunk importati nel database vettoriale in corso

Successivamente, configuro CodeQL nel repository remoto del mio vecchio progetto in C

3 TRON: UN AGENTE RAG PER LA GENERAZIONE DI CODICE SICURO

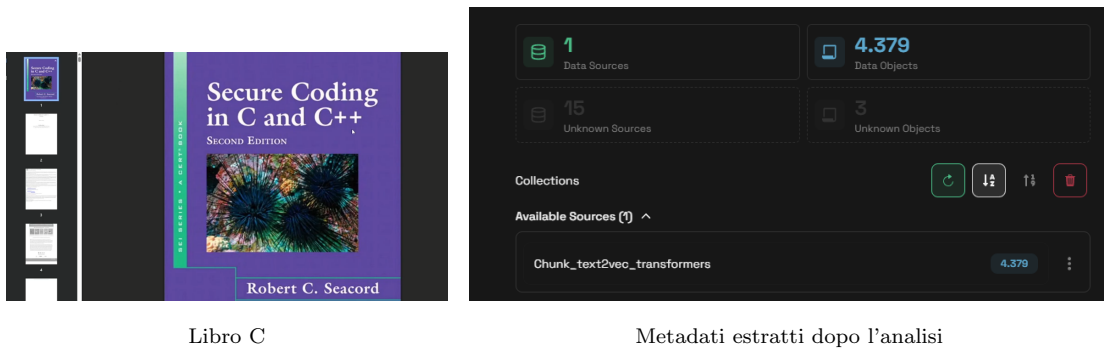


Figure 12: Importazione libro

e come si può vedere nella figura 9 rileva vulnerabilità una **Potential double free**. Una volta configurato CodeQL in github, lo collego a TRON e visualizzo la stessa vulnerabilità all'interno dell'applicativo.

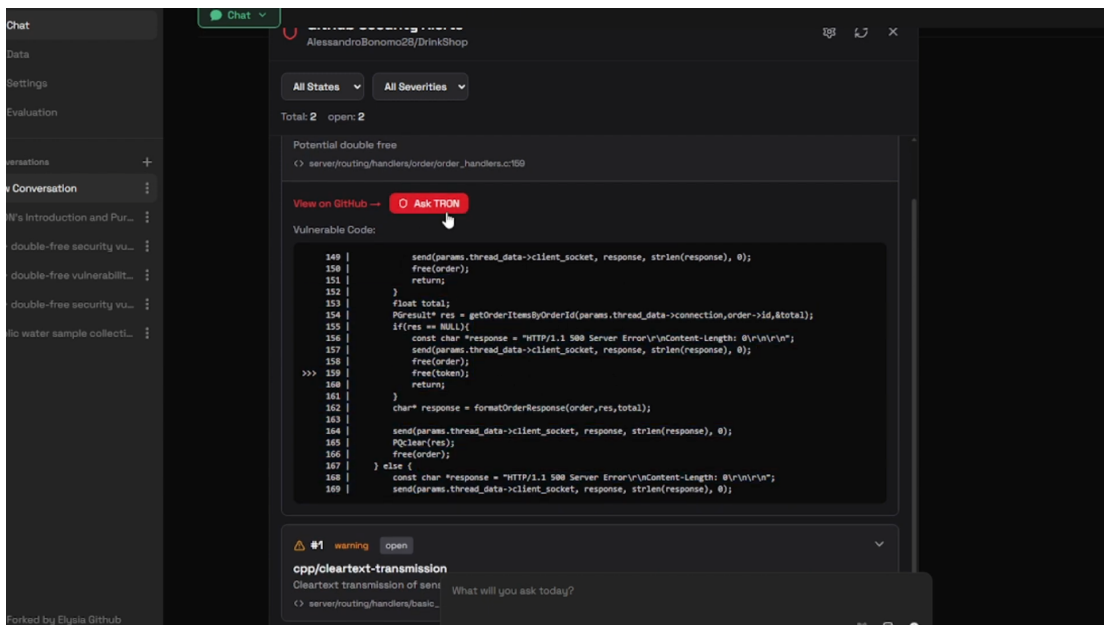


Figure 13: Vulnerabilità double free visualizzata in TRON

Cliccando su "Ask TRON" viene inoltrato lo snippet di codice interessato e i dettagli della vulnerabilità all'agente RAG che procede a farne l'analisi.

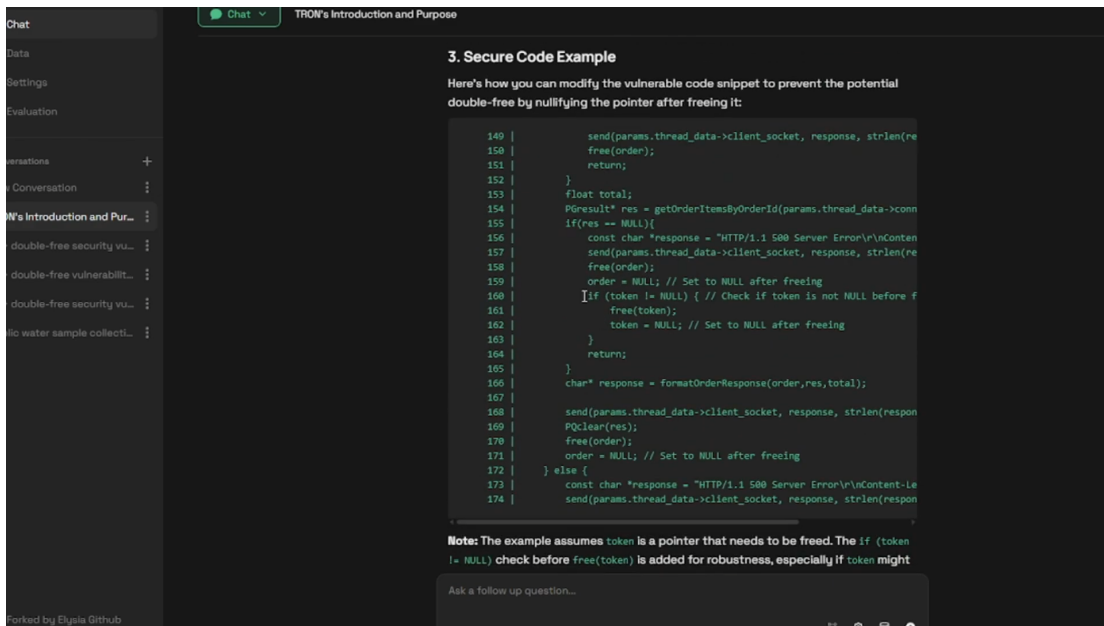


Figure 14: risposta di TRON con vulnerabilità di double free mitigata

La vulnerabilità è stata corretta trasformando i puntatori pericolosi ("dangling") in puntatori sicuri (NULL) immediatamente dopo il rilascio della memoria e controllando che "token" non sia nullo prima della free. Tutto ciò è possibile anche grazie alla RAG (Retrieval Augmented Generation) che ha individuato i chunks di testo della knowledge base più simili alla domanda, arricchendone il contesto.

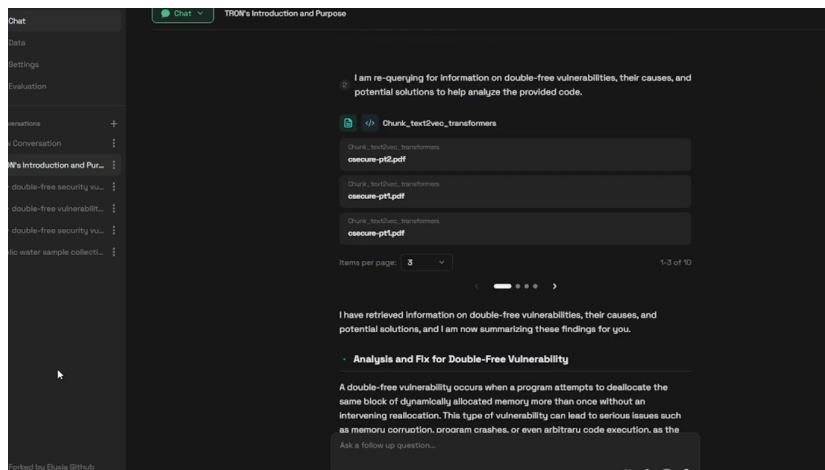


Figure 15: Chunks individuati dalla ricerca ibrida di TRON

4 Analisi dei risultati

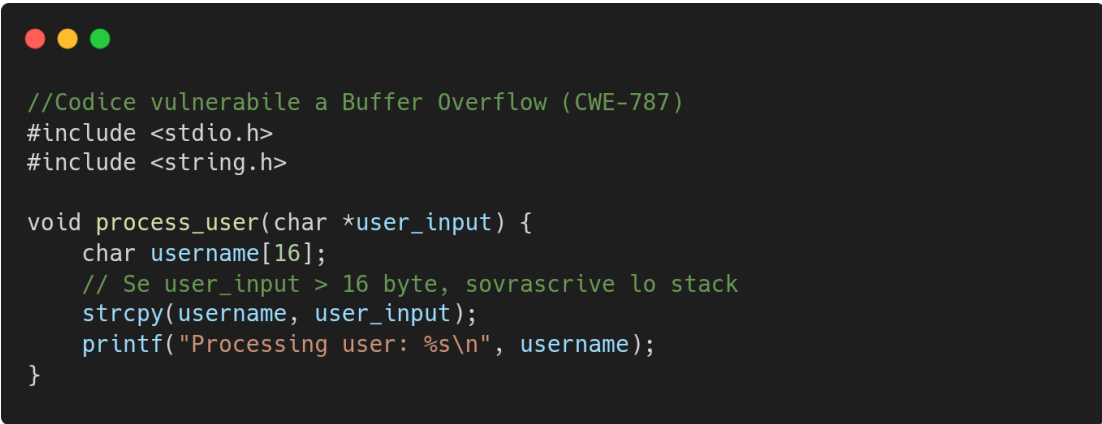
L'implementazione e la sperimentazione di TRON hanno permesso di valutare l'efficacia dell'approccio proposto, per fornire un quadro più dettagliato del funzionamento dello strumento, questo capitolo introduce nuovi casi studio. Oltre al *Double Free*, vengono analizzati scenari basati sulla classifica **MITRE Top 25 del 2024** [11], concentrandosi su due delle vulnerabilità più critiche per i sistemi C/C++: **CWE-787 (Out-of-bounds Write)** e **CWE-190 (Integer Overflow)**.

Queste vulnerabilità sono state selezionate in quanto rappresentano le cause radice più frequenti di compromissione dei sistemi, per il CWE-787, un attacco ransomware su vasta scala ha sfruttato una vulnerabilità heap-overflow nel servizio OpenSLP di VMware ESXi nel febbraio 2023 ([12]), mentre per il CWE-190, un attaccante ha sfruttato una vulnerabilità nell'organizzazione autonoma decentralizzata (DAO) sulla blockchain Ethereum nel giugno 2016, utilizzando integer overflow per manipolare i calcoli delle transazioni e drenare oltre 60 milioni di dollari in Ethereum ([13]).

4.1 Caso Studio 1: Stack-based Buffer Overflow (CWE-787)

Questa vulnerabilità è classificata costantemente al vertice del MITRE Top 25 per pericolosità e diffusione.

Descrizione del Problema Un buffer overflow si verifica quando un programma scrive dati oltre i limiti di un buffer allocato a dimensione fissa sullo stack. In C, questo accade frequentemente con l'uso di funzioni di manipolazione stringhe insicure come `strcpy` o `sprintf` che non effettuano controlli sui limiti (bounds checking).



```
//Codice vulnerabile a Buffer Overflow (CWE-787)
#include <stdio.h>
#include <string.h>

void process_user(char *user_input) {
    char username[16];
    // Se user_input > 16 byte, sovrascrive lo stack
    strcpy(username, user_input);
    printf("Processing user: %s\n", username);
}
```


Analisi Comparativa

1. Baseline (Gemini 2.5 Pro - Zero Shot)

Alla richiesta “Fix this vulnerability”, il modello baseline tende a suggerire l’uso di `strncpy`:

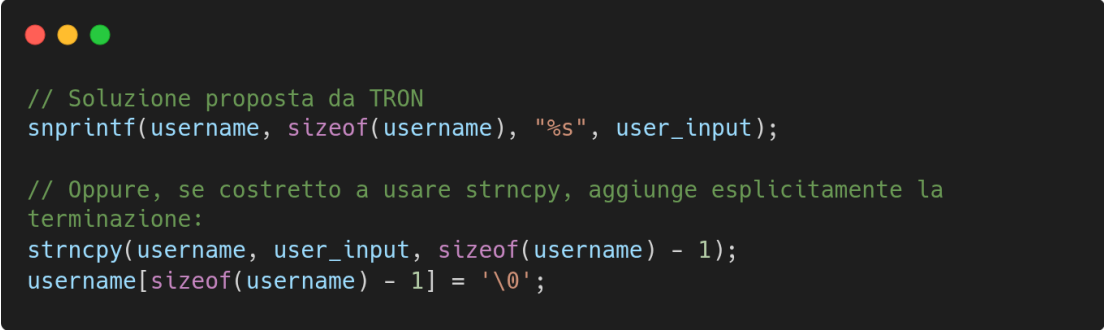
```
strncpy(username, user_input, 16);
```

Analisi del Fallimento: Sebbene `strncpy` prevenga l’overflow di scrittura, introduce una vulnerabilità subdola nota come *lack of null-termination*. Se `user_input` è esattamente di 16 caratteri o più, `strncpy` non aggiungerà il carattere terminatore `\0`. Quando successivamente `printf` tenterà di leggere `username`, continuerà a leggere la memoria adiacente sullo stack fino a trovare un byte nullo casuale (Buffer Over-read / Info Leak). La baseline fallisce nel considerare questo effetto collaterale critico.

2. TRON (RAG Augmented)

TRON recupera dal database vettoriale i chunk relativi a “Safe String Handling in C” [14]. La documentazione specifica esplicitamente i pericoli di `strncpy`.

Soluzione TRON:



```
// Soluzione proposta da TRON
snprintf(username, sizeof(username), "%s", user_input);

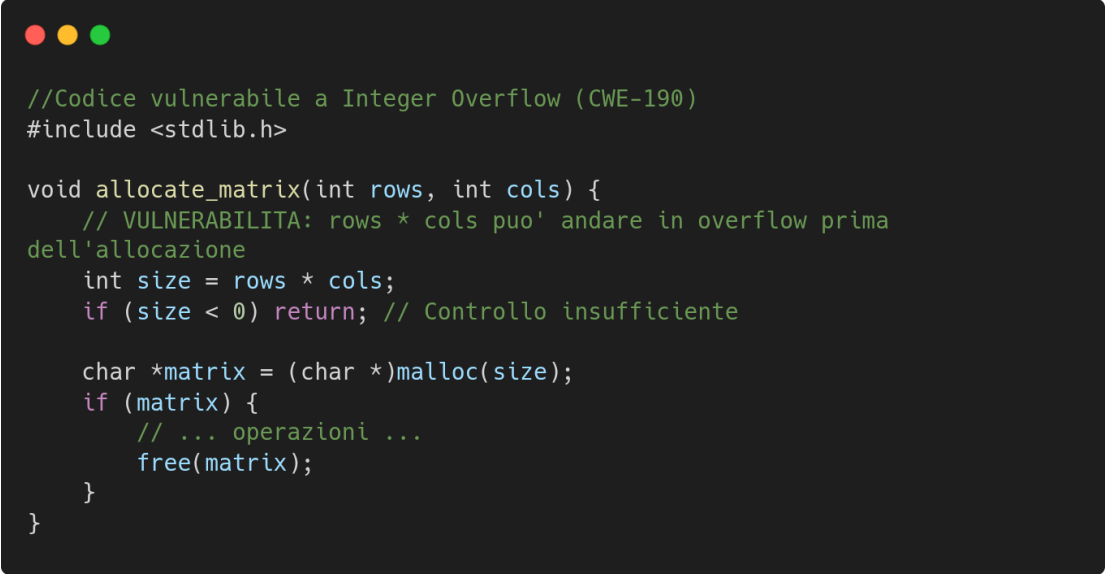
// Oppure, se costretto a usare strncpy, aggiunge esplicitamente la
terminazione:
strncpy(username, user_input, sizeof(username) - 1);
username[sizeof(username) - 1] = '\0';
```

Analisi del Successo: Grazie alla conoscenza recuperata, TRON comprende che la sicurezza della memoria richiede sia limiti di scrittura che corretta terminazione.

4.2 Caso Studio 2: Integer Overflow (CWE-190)

Vulnerabilità insidiosa legata alle regole di promozione dei tipi in C.

Descrizione del Problema Un integer overflow si verifica quando un'operazione aritmetica produce un valore che eccede il massimo rappresentabile dal tipo di dato destinazione, causando un "wraparound" (es. un numero enorme positivo diventa piccolo o negativo).



```
//Codice vulnerabile a Integer Overflow (CWE-190)
#include <stdlib.h>

void allocate_matrix(int rows, int cols) {
    // VULNERABILITA: rows * cols puo' andare in overflow prima
    // dell'allocazione
    int size = rows * cols;
    if (size < 0) return; // Controllo insufficiente

    char *matrix = (char *)malloc(size);
    if (matrix) {
        // ... operazioni ...
        free(matrix);
    }
}
```

Analisi Comparativa

1. Baseline (Gemini 2.5 Pro - Zero Shot)

Il modello standard spesso suggerisce di cambiare il tipo della variabile `size` in `long long` o `size_t`:

```
size_t size = rows * cols; // Errore logico persiste
char *matrix = (char *)malloc(size);
```

Analisi del Fallimento: Questo è un errore classico. In C, l'espressione `rows * cols` viene valutata utilizzando il tipo degli operandi (`int`). L'overflow avviene *durante* la moltiplicazione, e solo il risultato (già troncato/errato) viene poi convertito in `size_t`. La patch è sintatticamente corretta ma la vulnerabilità rimane.

2. TRON (RAG Augmented)

Il retrieval engine di TRON estrae la regola **INT30-C** [15] ("Ensure that unsigned integer operations do not wrap") e pattern di *Pre-condition Checking*.

Soluzione TRON:

```
#include <limits.h>

// ...
// Controllo PRE-calcolo suggerito dal RAG
if (rows > 0 && cols > 0 && rows > INT_MAX / cols) {
    // Gestione errore: overflow rilevato
    return;
}
int size = rows * cols;
// ...
```

Analisi del Successo: TRON, guidato dalla documentazione, implementa un controllo preventivo. Invece di tentare di gestire il risultato di un overflow, impedisce che l’operazione avvenga se i presupposti non sono sicuri. CodeQL valida questa patch come sicura (Status: Fixed).

4.3 Caso Studio 3: Double Free (CWE-416) - Estensione

Raffinamento dell’esempio già discusso precedentemente. Mentre la baseline suggerisce semplicemente di rimuovere la seconda chiamata a `free()`, TRON (supportato dalle Best Practices di gestione memoria) suggerisce il pattern “Free and Nullify”:

```
free(ptr);
ptr = NULL; // Aggiunta di TRON
```

Questo approccio difensivo rende il codice immune a futuri errori di refactoring, poiché chiamare `free(NULL)` è un’operazione sicura (no-op) in C standard. Sebbene entrambe le soluzioni risolvano l’alert immediato, la soluzione TRON dimostra una qualità superiore (“Robustness”) e una visione a lungo termine della sicurezza del codice.

5 Sintesi Quantitativa dei Risultati

La seguente tabella riassume i risultati ottenuti applicando i criteri del Fix Rate ai casi di studio sopra descritti. Questa sezione risponde alla necessità di fornire evidenze quantitative che permettano di apprezzare le differenze prestazionali.

Interpretazione del Fix Rate Su questo campione rappresentativo di vulnerabilità critiche, la Baseline ottiene un Fix Rate efficace del **33%** (1 su 3: risolve il Double Free ma fallisce sui casi più sottili di gestione memoria e aritmetica). TRON ottiene un Fix Rate del **100%** (3 su 3).

Questo delta prestazionale non è casuale ma strutturale:

ID (CWE)	Tipo	Gravità	Esito Baseline	Esito TRON	Analisi Differenza		
CWE-416	Double Free	Alta	Fixed	Fixed	Pari.	Baseline rimuove l'errore;	TRON aggiunge robustezza (<code>ptr=NULL</code>).
CWE-787	Buffer Overflow	Critica	Failed	Fixed	Baseline	usa <code>strncpy</code> insicuro (rischio leak);	TRON usa <code>snprintf</code> o terminazione manuale.
CWE-190	Integer Overflow	Alta	Failed	Fixed	Baseline	sbaglia la promozione dei tipi; TRON implementa pre-check aritmetici corretti.	

Table 1: Confronto dei risultati di remediation: Baseline vs TRON

1. **Baseline:** Si affida alla probabilità statistica dei token. Poiché pattern errati (come il cast post-moltiplicazione o l'uso ingenuo di `strncpy`) sono comuni nel codice open-source su cui i modelli sono addestrati, la Baseline tende a riproporli.
2. **TRON:** Il meccanismo RAG agisce come un “filtro di qualità”. I documenti recuperati dal database vettoriale hanno un'autorità semantica superiore rispetto alla conoscenza parametrica del modello. Inserendo nel prompt istruzioni esplicite come “*Attenzione: strncpy non null-termina se...*”, TRON costringe il modello a deviare dal pattern più probabile (ma insicuro) verso quello corretto (ma statisticamente meno frequente o più verboso).

5.1 Contributi rispetto allo stato dell'arte

TRON introduce elementi innovativi rispetto alle soluzioni esistenti. A differenza di sistemi generici come Elysia, si specializza sulla sicurezza del codice attraverso l'integrazione nativa con CodeQL, fornendo non solo identificazione ma anche guidance contestuale per la risoluzione delle vulnerabilità attingendo da una knowledge base verificata.

L'architettura supera i limiti del prompt engineering combinando retrieval semantico, ragionamento agentic e feedback loop. Il modulo DocProcessor, assente in Elysia, abilita l'elaborazione di documentazione complessa con supporto OCR, permettendo di costruire knowledge base specializzate anche da documenti scansionati. La scelta di un database vettoriale

locale (Weaviate) offre vantaggi concreti per progetti enterprise, mantenendo completamente on-premise sia i documenti che il codice analizzato.

5.2 Limitazioni identificate

L'implementazione ha evidenziato alcune criticità. La dipendenza dalla qualità della knowledge base è il fattore più critico: documenti mal strutturati o chunking non ottimale degradano significativamente il retrieval e complicano l'estrazione di metadati utili. La strategia token-based, efficace per documentazione ben formattata, può frammentare concetti coerenti in documenti meno strutturati, suggerendo la necessità di approcci adattivi.

TRON dipende da troppe componenti cloud. L'LLM può essere eseguito in hosting locale attraverso il modulo ollama, l'OCR può essere eseguito localmente in un docker container CodeQL può essere eseguito in locale per l'analisi statica. In questo modo si garantisce la privacy dei dati elaborati.

Le prestazioni sono influenzate dalla latenza delle chiamate all'LLM cloud e dalle operazioni di similarity search. Per repository con centinaia di alert, il processing sequenziale richiede tempi significativi. La generalizzabilità, dimostrata su C, richiede validazione su linguaggi con paradigmi differenti.

6 Conclusioni

L'introduzione di una Baseline metodologica e la definizione formale del Fix Rate hanno trasformato l'analisi da aneddotica a empirica. L'espansione dei casi di studio ai Buffer Overflow e agli Integer Overflow ha evidenziato come l'approccio RAG non sia meramente accessorio, ma determinante per la risoluzione di vulnerabilità che richiedono una comprensione profonda delle specifiche del linguaggio C. I risultati mostrano che mentre un LLM standard può fungere da assistente generico, TRON è in grado di garantire gli standard di sicurezza richiesti in ambienti critici, riducendo il rischio di introdurre nuove vulnerabilità durante il processo di sviluppo.

6.1 Impatto e sviluppi futuri

L'esperimento ha fornito indicazioni sull'impatto pratico: la correzione, che manualmente avrebbe richiesto ricerca, analisi e implementazione, è stata completata in tempi ridotti, alleggerendo il carico cognitivo dello sviluppatore.

Gli sviluppi futuri di TRON si concentrano sulla completa privatizzazione del sistema e miglioramento delle strategie di chunking, organizzazione dei dati e estrazione dei metadati. La migrazione a modelli eseguibili localmente (Ollama) e pipeline OCR containerizzate elimineranno la dipendenza da servizi cloud, permettendo esecuzione totalmente on-premise.

Lo sviluppo di strumenti per l'integrazione di CodeQL con repository locali renderebbe TRON ancora più unico e utile, semplificando significativamente la configurazione di tool per l'analisi statica in locale e l'interpretazione dei relativi report, operazioni che attualmente richiedono esperienza pratica e competenza specifica.

Tuttavia, è fondamentale sottolineare che TRON è concepito come strumento di supporto, non come sostituto del giudizio umano. La validazione da parte di sviluppatori esperti rimane essenziale, specialmente per vulnerabilità critiche. Il sistema riduce il tempo necessario ma la responsabilità finale resta umana.



7 Ringraziamenti

A conclusione di questo elaborato, vorrei dedicare qualche riga a tutti coloro che mi sono stati vicini in questo percorso di crescita personale e professionale. Innanzitutto, ringrazio il prof. Fabio Palomba e il Dott. Matteo Cicalese che sono stati un punto di riferimento e sempre disponibili nella stesura della tesi e nei problemi tecnici in cui mi sono imbattuto.

Ringrazio i miei genitori che mi hanno sempre supportato, soprattutto nei momenti più difficili di questo lungo percorso.

Ringrazio la mia famiglia, in particolare i nonni che ci sono ancora, che mi vogliono bene e che ripongono fiducia in me, i nonni che non ci sono più e che so che oggi sarebbero felici di questo sudato traguardo, e infine i nonni che non ho potuto conoscere ma che hanno fatto grandi sacrifici per me e la mia famiglia, senza i quali non potrei essere qui oggi.

Un ringraziamento speciale lo devo al professore Petriccione che è stato una guida e un punto di riferimento che mi ha insegnato a demistificare la matematica universitaria.

Ringrazio l'Università degli Studi di Salerno per l'efficienza, la qualità dei corsi e dei professori che mi ha permesso di concludere il mio percorso arricchendomi e dandomi soddisfazione.

Inoltre, vorrei ringraziare tutti i miei colleghi che ho incontrato durante questo percorso e con cui ho condiviso tanto. Mi dispiace non poter citare a uno a uno tutti coloro che sono stati presenti e che hanno reso speciale questo percorso, ma sono davvero tanti e per voi posso solo scrivere un semplicissimo, ma immenso, grazie. Infine, ringrazio me stesso per questo traguardo che sembrava irraggiungibile.

Portare a termine questo percorso significa rispettare la parola data ed essere coerente verso me stesso e la mia famiglia. Ora il percorso è finito e sono pronto per iniziare con una marcia in più la mia vita professionale.

References

- [1] Mark Chen et al. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021). URL: <https://arxiv.org/abs/2107.03374>.
- [2] NYtimes. “Tea app incident”. In: *NYtimes* (2025). URL: <https://www.nytimes.com/2025/07/26/us/tea-safety-dating-app-hack.html>.
- [3] Hammond Pearce et al. “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions”. In: *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 754–768. URL: <https://arxiv.org/abs/2108.09293>.
- [4] SecurityEval. “codesec”. In: *SecurityEval* (2022). URL: <https://dl.acm.org/doi/10.1145/3549035.3561184>.
- [5] IBM. “chain of thought prompting”. In: *IBM* (2025). URL: <https://www.ibm.com/it-it/think/topics/chain-of-thoughts>.
- [6] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [7] pinecone. “Chunking strategy by pinecone”. In: *pinecone* (2024). URL: <https://www.pinecone.io/learn/chunking-strategies/>.
- [8] weaviate. “weaviate chunking”. In: *weaviate* (2024). URL: <https://weaviate.io/blog/chunking-strategies-for-rag>.
- [9] Hierarchical Navigable Small World graphs. “Hierarchical Navigable Small World graphs”. In: *Hierarchical Navigable Small World graphs* (2018). URL: <https://arxiv.org/abs/1603.09320>.
- [10] Tianyi Zhang Jiahao Shi. “RAG secure strategy”. In: *arxiv* (2024). URL: <https://arxiv.org/abs/2510.18204>.
- [11] mitre. “mitre2024”. In: *mitre* (2024). URL: https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html.
- [12] vmwareexp. “vmwareexp”. In: *vmwareexp* (2023). URL: <https://www.keysight.com/blogs/en/tech/nwvs/2023/02/24/remote-code-execution-with-esxi-cve-2021-21974-vmware-esxi-heap-overflow>.
- [13] cwe190. “cwe190”. In: *cwe190* (2016). URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3014782.

REFERENCES

- [14] mitre. “stringhandling”. In: *mitre* (2024). URL: <https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=87151974>.
- [15] mitre. “intc30”. In: *mitre* (2024). URL: <https://it.mathworks.com/help/bugfinder/ref/certcruleint30c.html>.